SOFTWARE ENGINEERING · CHAPTER 06

Testing in Java Web Applications

Automated testing provides confidence that application behavior remains correct as the code evolves. JUnit supports test execution and assertions, while Mockito isolates units by creating test doubles.

JUnit 6.1 | Mockito 5.23 | Spring Boot 4.1 Test | AssertJ · Testcontainers · JaCoCo | JDK 25 LTS

AGENDA

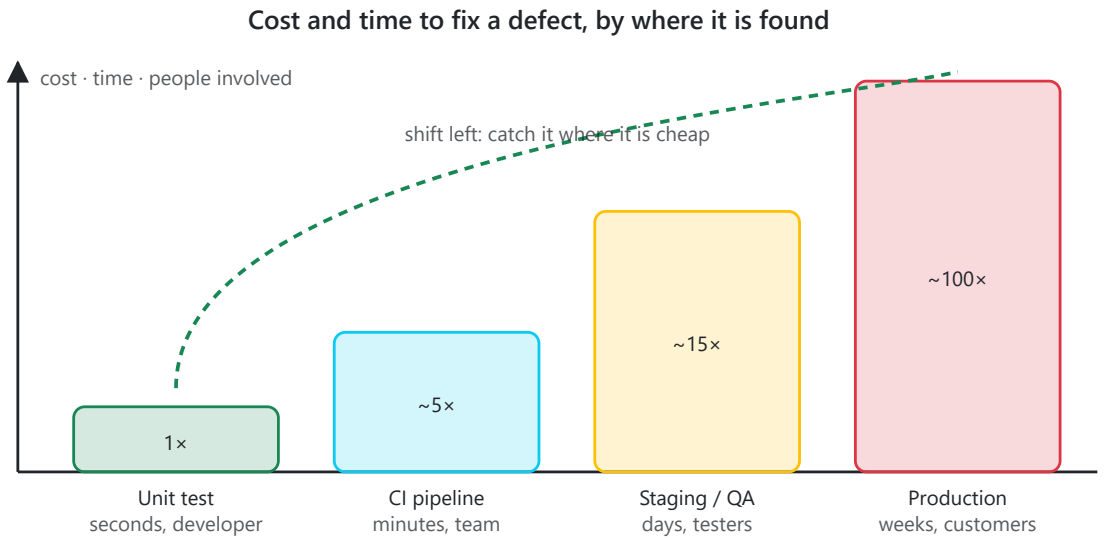
What we will cover

1. Test pyramid and test strategy
2. JUnit test structure, lifecycle, and assertions
3. Parameterized and repeated tests
4. Mockito mocks, stubs, spies, and verifications
5. Testing services, controllers, repositories, and filters
6. Test fixtures, builders, and test data management
7. Integration tests with databases and HTTP clients
8. Spring test contexts and slice tests
9. Contract, end-to-end, and regression testing
10. Code coverage, CI execution, and maintainable tests
11. Chapter Quiz
12. Lab-06

Click any item to jump directly to that topic.

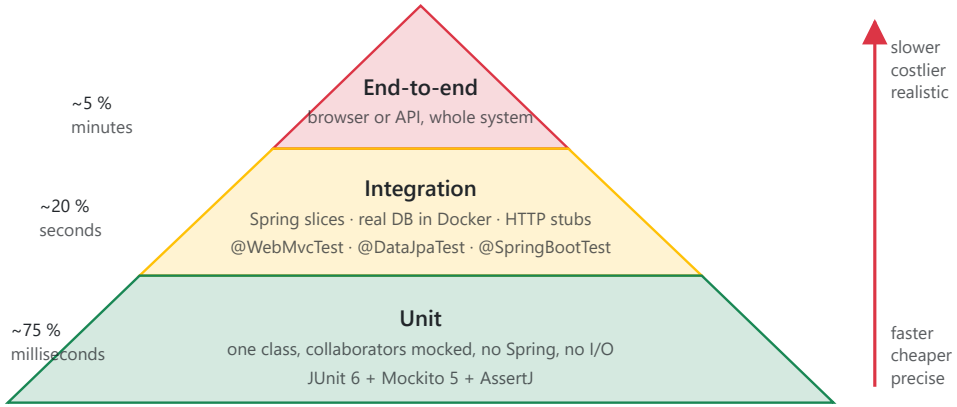
Why automated tests? Fast feedback, safe change

- A test is an **executable specification**: it states what the code must do and fails loudly when that stops being true.
- The later a defect is found, the more it costs: a failing unit test costs minutes, a production incident costs days and customers.
- Tests make **refactoring** safe (Chapter 02 layered design, Chapter 05 security rules) and document behavior for the next developer.
- Our toolbox in this chapter:
  - **JUnit 6.1**: the Platform runs tests, the Jupiter API declares them.
  - **Mockito 5.23**: test doubles that isolate the unit under test.
  - **AssertJ**: fluent, readable assertions.
  - **Spring Boot 4.1 test support**: slices, MockMvc, Testcontainers.
- Running example: the `edu.itc.shop` application (products, orders, JWT filter).



TOPIC 1

The test pyramid: many fast tests, few slow ones



Shop component	Where to test it	Why
`Money`, `PricingService`	Unit	pure logic, many edge cases, no dependencies
`ProductService`	Unit + mocks	rules depend on repository and price API answers
`ProductController`	`@WebMvcTest`	status codes, JSON shape, validation
`ProductRepository`	`@DataJpaTest` + Testcontainers	derived queries only fail against a real DB
`JwtFilter`	MockMvc with security	401/403 paths are configuration, not code
Whole API	`@SpringBootTest`, a few	wiring, serialization, transactions together

⚠ **Ice-cream cone** anti-pattern: mostly manual and end-to-end tests, few unit tests. Slow feedback, flaky builds, and a failure tells you only that "something" broke.

Test strategy in the build: dependencies and phases

```
<!-- Spring Boot project: one starter brings JUnit 6 Jupiter,
     Mockito 5, AssertJ, Hamcrest, JSONassert, Awaitility, Spring Test -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>

<!-- Plain Java module without Boot: declare the pieces yourself -->
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>6.1.3</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.23.0</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.assertj</groupId>
    <artifactId>assertj-core</artifactId>
    <version>3.27.6</version>
    <scope>test</scope>
</dependency>

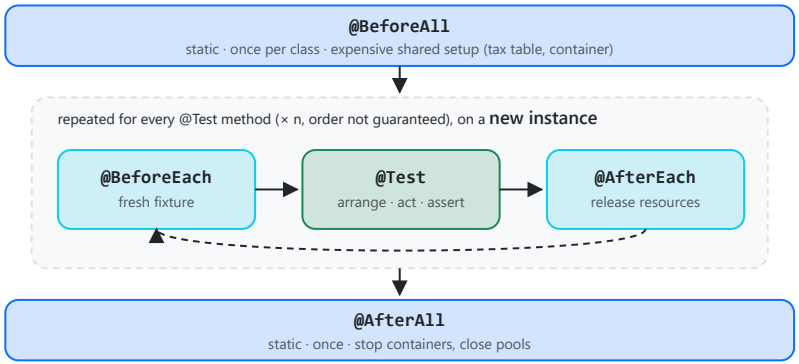
<!-- Surefire runs *Test in "test", Failsafe runs *IT in "verify".
     JDK 21+ warns when Mockito attaches its agent dynamically. -->
<plugin>
    <artifactId>maven-surefire-plugin</artifactId>
    <configuration>
        <argLine>@{argLine} -XX:+EnableDynamicAgentLoading</argLine>
    </configuration>
</plugin>
```

Part	Role
JUnit Platform 6	discovers and launches tests; IDEs, Maven Surefire and Gradle talk to it
JUnit Jupiter 6 (`org.junit.jupiter.api`)	the annotations and assertions you write
Mockito 5 (inline mock maker)	mocks, including final classes and records; static methods via `mockStatic`
AssertJ 3.27	fluent `assertThat(...)` with rich collection and exception support
Surefire / Failsafe	unit tests in `test`, integration tests in `integration-test` + `verify`

💡 **JUnit 6** requires Java 17+ and unified all module versions (Platform, Jupiter). The Vintage engine for JUnit 4 tests is gone: migrate old tests to Jupiter (`@Before` → `@BeforeEach`, `@RunWith` → `@ExtendWith`).

⚠ Strategy is also naming: `*Test` = fast, always run; `*IT` = needs Docker or network, run in `verify` and in CI.

JUnit Jupiter: test class structure and lifecycle



Annotation	Meaning
`@DisplayName`	human-readable name in reports
`@Nested`	inner class grouping a context ("when …")
`@Disabled("reason")`	skip, with a reason that is reported
`@Tag("slow")`	filter groups from Maven or the IDE
`@Timeout(2)`	fail if the test exceeds 2 s

```
package edu.itc.shop.service;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("PricingService")
class PricingServiceTest {

    static TaxTable taxes;        // shared, loaded once
    PricingService pricing;       // fresh for every test

    @BeforeAll
    static void loadTaxTable() { taxes = TaxTable.load("eu-2026"); }

    @BeforeEach
    void setUp() { pricing = new PricingService(taxes); }

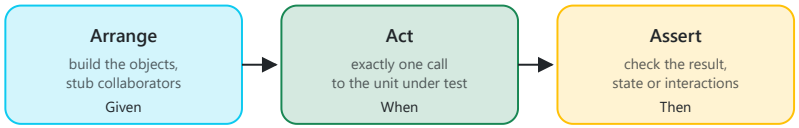
    @Test
    @DisplayName("adds 19 % German VAT to a net price")
    void addsVat() {
        Money gross = pricing.gross(Money.of("10.00"), "DE");
        assertEquals(Money.of("11.90"), gross);
    }

    @Nested
    @DisplayName("when the country is unknown")
    class UnknownCountry {
        @Test
        void throwsWithAClearMessage() {
            var ex = assertThrows(IllegalArgumentException.class,
                () -> pricing.gross(Money.of("10.00"), "XX"));
            assertTrue(ex.getMessage().contains("XX"));
        }
    }

    @AfterEach
    void tearDown() { pricing = null; }

    @AfterAll
    static void done() { taxes = null; }
}
```

Assertions: Arrange, Act, Assert with JUnit and AssertJ



One behavior per test: the name says what, the assertion says how much.

A failing test should point at one cause, not at "something in the order flow".

JUnit `Assertions`	AssertJ
`assertEquals(exp, act)`	`assertThat(act).isEqualTo(exp)`
`assertThrows(E.class, () -> …)`	`assertThatThrownBy(() -> …)`
`assertAll(…)` groups checks	chained calls, soft assertions
`assertIterableEquals`	`containsExactly`, `extracting`
`assertTimeout(Duration, …)`	Awaitility for async

```java
@Test
void orderTotalSumsLinesAndAppliesDiscount() {
    // Arrange
    Order order = new Order(customer("alice"));
    order.add(product("Keyboard", "49.90"), 2);
    order.add(product("Mouse", "19.90"), 1);
    order.applyDiscount(Percent.of(10));
    // Act
    Money total = order.total();
    // Assert: all three checks run, failures are reported together
    assertAll("total",
        () -> assertEquals(Money.of("107.73"), total),
        () -> assertEquals(3, order.itemCount()),
        () -> assertTrue(order.hasDiscount()));
}

@Test
void assertjReadsLikeASentence() {
    List<OrderLine> lines = order.lines();

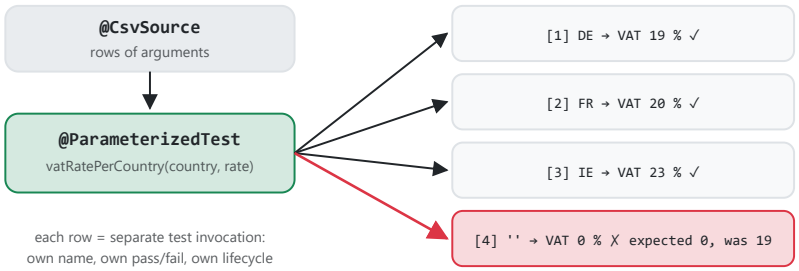
    assertThat(lines)
        .hasSize(2)
        .extracting(OrderLine::productName, OrderLine::quantity)
        .containsExactly(tuple("Keyboard", 2), tuple("Mouse", 1));

    assertThat(order.total().amount())
        .isEqualByComparingTo("107.73");   // BigDecimal scale-safe

    assertThatThrownBy(() -> order.add(null, 1))
        .isInstanceOf(NullPointerException.class)
        .hasMessageContaining("product");
}
```

Expected value first in JUnit (`assertEquals(expected, actual)`); AssertJ avoids the mix-up because the actual value is always inside `assertThat(...)`.

Parameterized tests: one method, many cases



Reports show exactly which case broke instead of stopping at the first failed loop iteration.

```java
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;

@ParameterizedTest(name = "[{index}] {0} → VAT {1} %")
@CsvSource({
    "DE, 19",
    "FR, 20",
    "IE, 23",
    "'', 0"           // quoted empty string: no country, no tax
})
void vatRatePerCountry(String country, int expectedRate) {
    assertEquals(expectedRate, taxes.rateFor(country));
}

@ParameterizedTest
@ValueSource(strings = {"", "  ", "\t"})
void blankSkuIsRejected(String sku) {
    assertThrows(IllegalArgumentException.class, () -> new Sku(sku));
}

@ParameterizedTest
@NullAndEmptySource                 // null, "" ...
@ValueSource(strings = {" "})       // ... plus a blank
void blankProductNameIsRejected(String name) {
    assertThatThrownBy(() -> new Product("KB-01", name, Money.ZERO))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("name");
}

@ParameterizedTest
@CsvSource(textBlock = """
    net,     country, gross
    10.00,   DE,      11.90
    100.00,  FR,      120.00
    """, useHeadersInDisplayName = true)
void grossPrice(String net, String country, String gross) {
    assertEquals(Money.of(gross), pricing.gross(Money.of(net), country));
}
```

Richer sources and repeated tests

```java
@ParameterizedTest
@MethodSource("discountCases")
void discountIsApplied(Money net, Percent off, Money expected) {
    assertEquals(expected, pricing.discounted(net, off));
}
static Stream<Arguments> discountCases() {
    return Stream.of(
        arguments(Money.of("100.00"), Percent.of(0),  Money.of("100.00")),
        arguments(Money.of("100.00"), Percent.of(15), Money.of("85.00")),
        arguments(Money.of("0.01"),   Percent.of(50), Money.of("0.01")));
}

@ParameterizedTest
@EnumSource(value = OrderStatus.class, names = {"PAID", "SHIPPED"})
void paidOrdersCannotBeEdited(OrderStatus status) {
    assertFalse(status.isEditable());
}

@ParameterizedTest
@EnumSource(value = OrderStatus.class, mode = EnumSource.Mode.EXCLUDE,
            names = {"PAID", "SHIPPED"})
void otherOrdersCanBeEdited(OrderStatus status) {
    assertTrue(status.isEditable());
}

// a random-based generator: run it many times, keep what you saw
private static final Set<String> seen = ConcurrentHashMap.newKeySet();

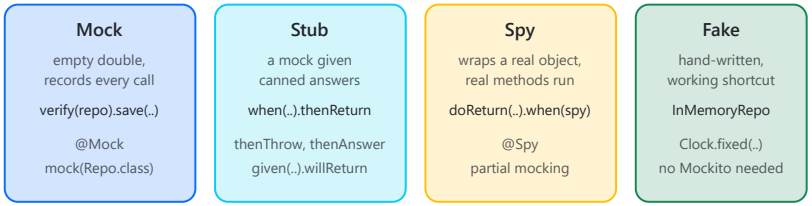
@RepeatedTest(value = 50, name = "id {currentRepetition}/{totalRepetitions}")
void generatedIdsAreUniqueAndWellFormed(RepetitionInfo info) {
    String id = OrderId.generate();
    assertTrue(id.matches("ORD-[0-9A-Z]{8}"), id);
    assertTrue(seen.add(id), "duplicate id " + id
        + " in repetition " + info.getCurrentRepetition());
}
```

Source	Provides
`@ValueSource`	one literal per invocation (ints, strings, classes…)
`@CsvSource` / `@CsvFileSource`	rows of simple values, inline or from `src/test/resources`
`@MethodSource`	a static factory returning `Stream<Arguments>`: any object type
`@EnumSource`	all constants or a filtered subset (`names`, `mode`)
`@NullSource`, `@EmptySource`	the classic edge cases
`@ArgumentsSource`	your own provider class
`@ParameterizedClass`	parameterize a whole class (JUnit 5.13+/6)

💡 Implicit conversion turns `"19"` into `int`, `"2026-09-25"` into `LocalDate`, `"PAID"` into an enum. For your own types add a static factory or an `ArgumentConverter`.

⚠ `@RepeatedTest` is for probing randomness, concurrency or flakiness, not for "running it again so it passes". A repeated test that fails once is a real failure.

Mockito 5: test doubles that isolate the unit under test



Mockito creates mocks, stubs and spies; fakes you write yourself.

Mock what you own and what is slow or non-deterministic (DB, HTTP, clock, random).
Do not mock value objects (Money) or the class under test.

```java
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock ProductRepository products;
    @Mock PaymentGateway payments;
    @InjectMocks OrderService orders;   // constructor injection of the mocks

    @Test
    void placesOrderWhenStockIsAvailable() {
        // stub: what the collaborators answer
        when(products.findBySku("KB-01"))
            .thenReturn(Optional.of(product("KB-01", "49.90", 5)));
        when(payments.charge(any(Money.class)))
            .thenReturn(new Receipt("rc-1"));

        Order order = orders.place("alice", "KB-01", 2);

        assertEquals(OrderStatus.PAID, order.status());
        // verify: the interaction that matters
        verify(payments).charge(Money.of("99.80"));
        verifyNoMoreInteractions(payments);
    }

    @Test
    void failsWhenPaymentIsDeclined() {
        when(products.findBySku("KB-01"))
            .thenReturn(Optional.of(product("KB-01", "49.90", 5)));
        when(payments.charge(any()))
            .thenThrow(new PaymentDeclinedException("card expired"));

        assertThrows(PaymentDeclinedException.class,
            () -> orders.place("alice", "KB-01", 2));
        verify(products, never()).save(any());
    }
}
```

Verifications in depth: captors, order, spies, strict stubs

```java
import static org.mockito.BDDMockito.*;   // given / then aliases

@Captor ArgumentCaptor<Order> saved;
@Spy MoneyFormatter formatter = new MoneyFormatter(Locale.GERMANY);

@Test
void savesOrderWithComputedTotalAfterCharging() {
    given(products.findBySku("KB-01"))
        .willReturn(Optional.of(product("KB-01", "49.90", 5)));
    given(payments.charge(any())).willReturn(new Receipt("rc-1"));

    orders.place("alice", "KB-01", 2);

    // capture the argument and assert on it
    then(products).should(times(1)).save(saved.capture());
    assertThat(saved.getValue().total()).isEqualTo(Money.of("99.80"));
    assertThat(saved.getValue().receiptId()).isEqualTo("rc-1");

    // order of interactions across mocks
    InOrder inOrder = inOrder(payments, products);
    inOrder.verify(payments).charge(any());
    inOrder.verify(products).save(any());
}

@Test
void spyKeepsRealBehaviorUnlessStubbed() {
    // real method runs
    assertEquals("99,80 €", formatter.format(Money.of("99.80")));
    // partial stub: doReturn avoids calling the real method while stubbing
    doReturn("n/a").when(formatter).format(Money.ZERO);
    assertEquals("n/a", formatter.format(Money.ZERO));
    verify(formatter, times(2)).format(any());
}

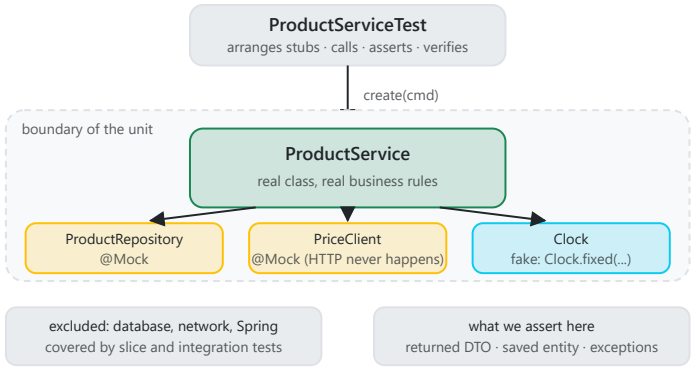
@Test
void unusedStubbingFailsUnderStrictStubs() {
    // MockitoExtension defaults to STRICT_STUBS: this stub is never used,
    // so the test fails with UnnecessaryStubbingException
    when(products.findBySku("unused")).thenReturn(Optional.empty());
    assertNotNull(orders);
}
```

Tool	Use it for
`verify(m, times(n))`, `never()`, `atLeastOnce()`	how often a call happened
`ArgumentCaptor`, `argThat(p -> …)`	inspect what was passed
`InOrder`	sequence across one or more mocks
`thenAnswer(inv -> …)`	compute the answer from arguments (echo a saved entity)
`doReturn/doThrow(..).when(spy)`	stub spies and void methods
`mockStatic(Clazz.class)`	static methods, inline mock maker; prefer injecting a `Clock`
`lenient()`, `@MockitoSettings(strictness = LENIENT)`	opt out of strict stubs for shared setup

⚠ **Over-verification** couples the test to the implementation. Verify interactions that are the point of the behavior (a payment was charged, an e-mail was sent); assert on return values and state for everything else.

Mockito 5 uses the *inline* mock maker by default: final classes, records and enums can be mocked, but if you feel the need, reconsider the design first.

Testing a service: the unit and its boundary




```java
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {
    @Mock ProductRepository repo;
    @Mock PriceClient prices;
    Clock clock = Clock.fixed(Instant.parse("2026-09-25T10:00:00Z"),
                              ZoneOffset.UTC);
    ProductService service;

    @BeforeEach
    void setUp() { service = new ProductService(repo, prices, clock); }

    @Test
    void createStoresProductWithCurrentPriceAndTimestamp() {
        when(prices.currentPrice("KB-01")).thenReturn(Money.of("49.90"));
        // echo back whatever the service saves (id assigned by DB later)
        when(repo.save(any(Product.class)))
            .thenAnswer(inv -> inv.getArgument(0));

        ProductDto dto = service.create(new CreateProduct("KB-01", "Keyboard"));

        assertThat(dto.price()).isEqualTo(Money.of("49.90"));
        assertThat(dto.createdAt()).isEqualTo(clock.instant());
        verify(repo).save(argThat(p -> p.sku().equals("KB-01")));
    }

    @Test
    void getThrowsNotFoundWithTheIdInTheMessage() {
        when(repo.findById(42L)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> service.get(42L))
            .isInstanceOf(ProductNotFoundException.class)
            .hasMessage("Product 42 not found");
        verifyNoInteractions(prices);
    }
}
```

Inject a `Clock` instead of calling `Instant.now()`: time becomes a fake you control, and no `mockStatic` is needed.

Testing a controller: `@WebMvcTest`, MockMvc and `@MockitoBean`

```java
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.assertj.MockMvcTester;

@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired MockMvcTester mvc;      // AssertJ-style MockMvc (Spring 6.2+)
    @MockitoBean ProductService service;   // replaces the bean in the slice

    @Test
    @WithMockUser
    void listReturnsJsonArray() {
        given(service.search(null)).willReturn(List.of(
            new ProductDto(1L, "KB-01", "Keyboard", Money.of("49.90"))));

        assertThat(mvc.get().uri("/api/products"))
            .hasStatusOk()
            .hasContentType(MediaType.APPLICATION_JSON)
            .bodyJson()
            .extractingPath("$[0].name").isEqualTo("Keyboard");
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void createWithBlankNameReturns400ProblemDetail() {
        assertThat(mvc.post().uri("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                    {"sku": "KB-01", "name": ""}
                    """))
            .hasStatus(HttpStatus.BAD_REQUEST)
            .hasContentType(MediaType.APPLICATION_PROBLEM_JSON)
            .bodyJson()
            .extractingPath("$.errors[0].field").isEqualTo("name");
        then(service).shouldHaveNoInteractions();
    }

    @Test
    void anonymousIsRejected() {
        assertThat(mvc.get().uri("/api/products"))
            .hasStatus(HttpStatus.UNAUTHORIZED);
    }
}
```

`@WebMvcTest` loads	Does not load
`@Controller`, `@RestControllerAdvice`	`@Service`, `@Repository`
Jackson converters, validation	DataSource, JPA
`Filter` beans, Spring Security config	Scheduled jobs, listeners
`MockMvc` / `MockMvcTester`	a real HTTP server

Classic `MockMvc`	`MockMvcTester`
`mvc.perform(get(..)).andExpect(status().isOk())`	`assertThat(mvc.get().uri(..)).hasStatus(`
Hamcrest matchers, checked `Exception`	AssertJ chain, no checked exception
`jsonPath("$.x").value(..)`	`bodyJson().extractingPath("$.x")`

In Spring Boot 4 the slice lives in module `spring-boot-webmvc-test` (pulled in by the test starter). `@MockitoBean` replaced `@MockBean`; `@MockitoSpyBean` replaced `@SpyBean`.

⚠ Security is active in the slice: without `@WithMockUser` you test the 401 path, which is a useful test in itself. Import your Chapter 05 `SecurityConfig` to test the real rules.

Testing a repository (`@DataJpaTest`) and a security filter

```java
import org.springframework.boot.data.jpa.test.autoconfigure
        .DataJpaTest;
import org.springframework.boot.data.jpa.test.autoconfigure
        .TestEntityManager;

@DataJpaTest
class ProductRepositoryTest {
    @Autowired TestEntityManager em;
    @Autowired ProductRepository repo;

    @Test
    void findsByNameIgnoringCase() {
        em.persist(new Product("KB-01", "Mechanical Keyboard", 4990));
        em.persist(new Product("MS-01", "Mouse", 1990));
        em.flush();
        em.clear();   // force a real SELECT, not a cache hit

        List<Product> found =
            repo.findByNameContainingIgnoreCase("keyboard");

        assertThat(found).extracting(Product::sku)
                         .containsExactly("KB-01");
    }

    @Test
    void skuIsUnique() {
        em.persistAndFlush(new Product("KB-01", "A", 1));
        assertThatThrownBy(() ->
                em.persistAndFlush(new Product("KB-01", "B", 2)))
            .isInstanceOf(PersistenceException.class);
    }
}
```

💡 Each `@DataJpaTest` method runs in a transaction that is rolled back; an embedded database replaces yours unless a Testcontainers `@ServiceConnection` provides one.

```java
@WebMvcTest(ProductController.class)
@Import({SecurityConfig.class, JwtFilter.class})   // Chapter 05
class JwtFilterTest {
    @Autowired MockMvc mvc;
    @MockitoBean ProductService service;
    @MockitoBean JwtService jwt;

    @Test
    void missingTokenIs401() throws Exception {
        mvc.perform(get("/api/products"))
           .andExpect(status().isUnauthorized());
        then(jwt).shouldHaveNoInteractions();
    }

    @Test
    void validTokenPopulatesSecurityContext() throws Exception {
        given(jwt.parse("good")).willReturn(
            new JwtClaims("alice", List.of("ROLE_USER")));

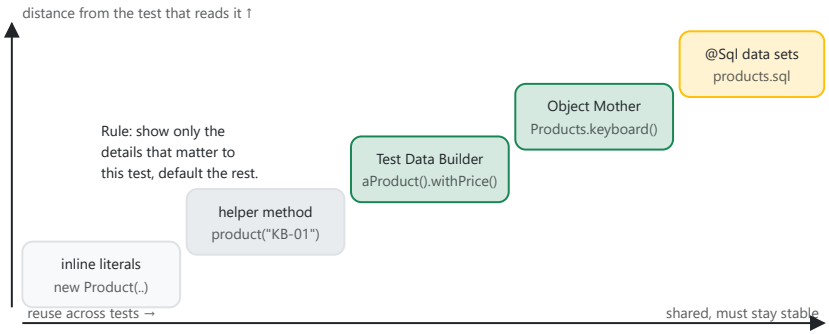
        mvc.perform(get("/api/products")
                .header(HttpHeaders.AUTHORIZATION, "Bearer good"))
           .andExpect(status().isOk())
           .andExpect(authenticated().withUsername("alice"));
    }

    @Test
    void expiredTokenIs401WithProblemDetail() throws Exception {
        given(jwt.parse("old")).willThrow(new JwtException("expired"));

        mvc.perform(get("/api/products")
                .header(HttpHeaders.AUTHORIZATION, "Bearer old"))
           .andExpect(status().isUnauthorized())
           .andExpect(jsonPath("$.detail").value("expired"));
    }
}
```

⚠ For the filter's pure logic (header parsing) a plain unit test with `MockHttpServletRequest` and `MockFilterChain` is faster; use MockMvc for the wiring into the security chain.

Fixtures that stay readable: builders and the Object Mother




```java
// src/test/java/edu/itc/shop/testing/ProductBuilder.java
public final class ProductBuilder {
    private static final AtomicInteger seq = new AtomicInteger();
    private String sku = "SKU-" + seq.incrementAndGet();  // unique default
    private String name = "Any product";
    private Money price = Money.of("10.00");
    private int stock = 10;

    public static ProductBuilder aProduct() { return new ProductBuilder(); }
    public ProductBuilder withSku(String s)   { sku = s; return this; }
    public ProductBuilder withName(String n)  { name = n; return this; }
    public ProductBuilder withStock(int s)    { stock = s; return this; }
    public ProductBuilder outOfStock()        { return withStock(0); }
    public ProductBuilder withPrice(String p) {
        price = Money.of(p); return this;
    }
    public Product build() { return new Product(sku, name, price, stock); }
}

// Object Mother: named scenarios the whole team recognises
public final class Products {
    public static Product keyboard() {
        return aProduct().withSku("KB-01").withName("Mechanical Keyboard")
                         .withPrice("49.90").build();
    }
    public static Product soldOutMouse() {
        return aProduct().withSku("MS-01").withName("Mouse")
                         .outOfStock().build();
    }
}

// in a test only the relevant detail is visible
Product cheapest = aProduct().withPrice("0.01").build();
Order order = anOrder().with(Products.keyboard(), 2).paid().build();
```

Database fixtures with `@Sql` and keeping tests independent

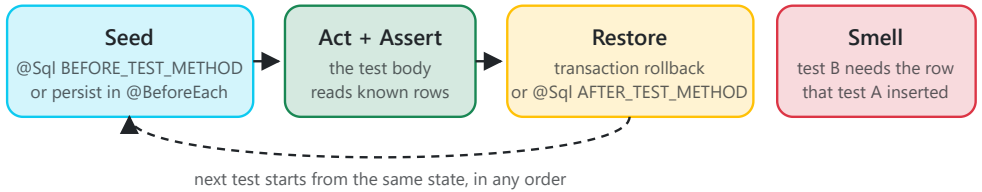
```java
@DataJpaTest
@Sql(scripts = "/fixtures/products.sql")          // before each test
@Sql(scripts = "/fixtures/cleanup.sql",
     executionPhase = Sql.ExecutionPhase.AFTER_TEST_METHOD)
class ProductQueryTest {
    @Autowired ProductRepository repo;

    @Test
    void countsCheapProducts() {
        assertThat(repo.countByPriceCentsLessThan(2000)).isEqualTo(2);
    }

    @Test
    @Sql(statements = "UPDATE product SET stock = 0")   // per-test tweak
    void findsNothingInStock() {
        assertThat(repo.findByStockGreaterThan(0)).isEmpty();
    }
}
```

```sql
-- src/test/resources/fixtures/products.sql
INSERT INTO product (id, sku, name, price_cents, stock) VALUES
  (1, 'KB-01', 'Mechanical Keyboard', 4990, 5),
  (2, 'MS-01', 'Mouse',               1990, 0),
  (3, 'CB-01', 'USB-C cable',          790, 25);

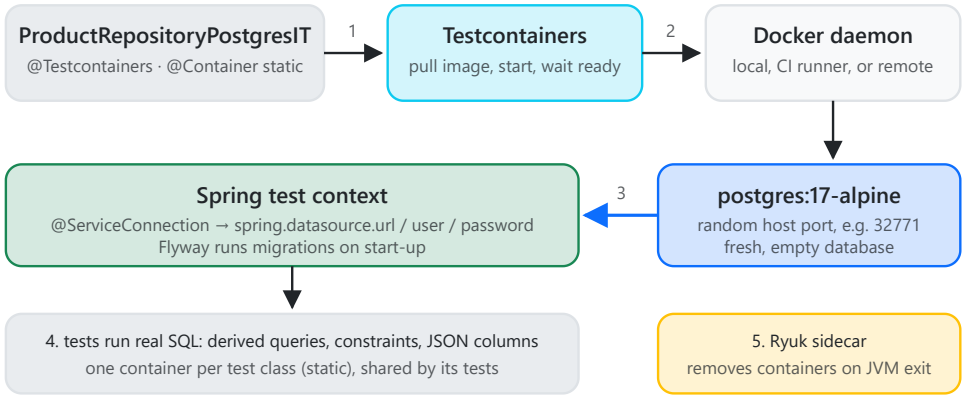
-- src/test/resources/fixtures/cleanup.sql
DELETE FROM order_line; DELETE FROM orders; DELETE FROM product;
```



Rollback is fastest; explicit cleanup is needed when the code under test commits (REQUIRES_NEW, async).

Strategy	Good for	Watch out
Persist in the test / builders	unit-like repository tests	verbose for large graphs
`@Sql` scripts	reference data, complex queries	schema drift, hard-coded ids
Flyway migrations + seed profile	integration tests on real DB	keep seeds tiny
Random data with a fixed seed	property-style tests	log the seed on failure

Integration tests with a real database: Testcontainers



Same engine as production (PostgreSQL), not an H2 approximation: behavior differences surface in CI, not in prod.

```java
import org.springframework.boot.testcontainers.service.connection
        .ServiceConnection;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.*;

@SpringBootTest
@Testcontainers
class ProductRepositoryPostgresIT {

    @Container
    @ServiceConnection          // no spring.datasource.* properties needed
    static PostgreSQLContainer postgres =
        new PostgreSQLContainer("postgres:17-alpine");

    @Autowired ProductRepository repo;

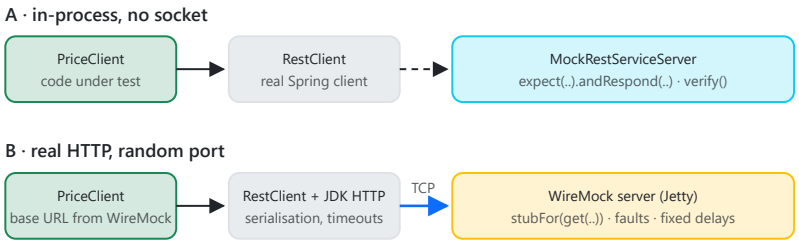
    @Test
    void derivedQueryRunsOnRealPostgres() {
        repo.save(Products.keyboard());

        assertThat(repo.findByNameContainingIgnoreCase("KEYBOARD"))
            .extracting(Product::sku).containsExactly("KB-01");
    }
}
```

```xml
<!-- versions managed by the Spring Boot 4.1 BOM -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-postgresql</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
```

Works with `@DataJpaTest` too: since Boot 3.4 the slice keeps a `@ServiceConnection` DataSource instead of replacing it with an embedded one. No Docker on the machine? Provide a fallback profile (Lab-06, Task 5).

Integration tests for HTTP clients: `MockRestServiceServer` and WireMock



A is faster and Spring-aware; B also exercises the transport and can simulate timeouts and connection resets.

```java
// slice: the client bean, a RestClient.Builder and a bound mock server
@RestClientTest(PriceClient.class)
class PriceClientTest {
    @Autowired PriceClient client;
    @Autowired MockRestServiceServer server;

    @Test
    void parsesPriceFromPaymentApi() {
        server.expect(requestTo("/prices/KB-01"))
              .andExpect(method(HttpMethod.GET))
              .andExpect(header(HttpHeaders.ACCEPT, containsString("json")))
              .andRespond(withSuccess("""
                  {"sku":"KB-01","amount":"49.90","currency":"EUR"}
                  """, MediaType.APPLICATION_JSON));

        assertThat(client.currentPrice("KB-01")).isEqualTo(Money.of("49.90"));
        server.verify();              // every expectation was met
    }

    @Test
    void mapsServerErrorToPriceUnavailable() {
        server.expect(requestTo("/prices/KB-01")).andRespond(withServerError());

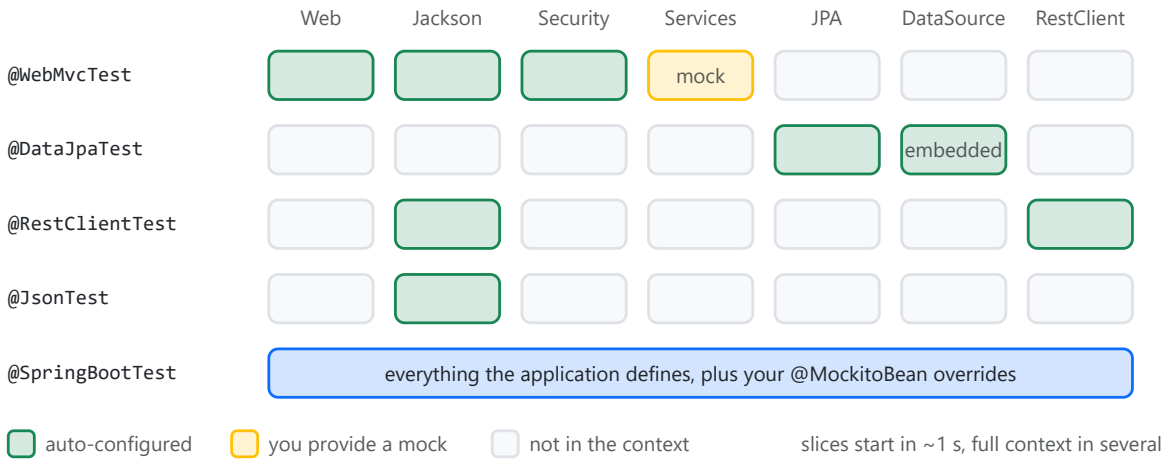
        assertThatThrownBy(() -> client.currentPrice("KB-01"))
            .isInstanceOf(PriceUnavailableException.class);
    }
}
```

```java
@WireMockTest                          // org.wiremock:wiremock 3.x
class PriceClientWireMockTest {
    @Test
    void talksRealHttp(WireMockRuntimeInfo wm) {
        stubFor(get("/prices/KB-01").willReturn(okJson("""
            {"sku":"KB-01","amount":"49.90","currency":"EUR"}
            """).withFixedDelay(200)));
        var client = new PriceClient(RestClient.create(wm.getHttpBaseUrl()));

        assertThat(client.currentPrice("KB-01")).isEqualTo(Money.of("49.90"));
    }
}
```

TOPIC 8

Spring test contexts: slices load a part, `@SpringBootTest` loads it all




```java
@JdbcTest                       // JdbcClient + embedded DB
@Import(JdbcProductRepository.class)
class JdbcProductRepositoryTest {
    @Autowired JdbcProductRepository repo;
    @Autowired JdbcClient jdbc;

    @Test
    void insertsAndReads() {
        repo.save(Products.keyboard());
        Integer n = jdbc.sql("select count(*) from product")
                        .query(Integer.class).single();
        assertThat(n).isEqualTo(1);
    }
}
```

- A slice = `@ExtendWith(SpringExtension)` + a filtered component scan + matching auto-configurations. Add beans with `@Import`, replace them with `@MockitoBean`.
- `@SpringBootTest` starts from `@SpringBootApplication`; default `webEnvironment = MOCK` (no port, MockMvc available with `@AutoConfigureMockMvc`).
- `@TestConfiguration` classes add test-only beans (a fixed `Clock`) without being picked up by the main scan.
- `@ActiveProfiles("test")` + `application-test.yml` for test-only properties.

Full-stack test on a random port, and why context caching matters

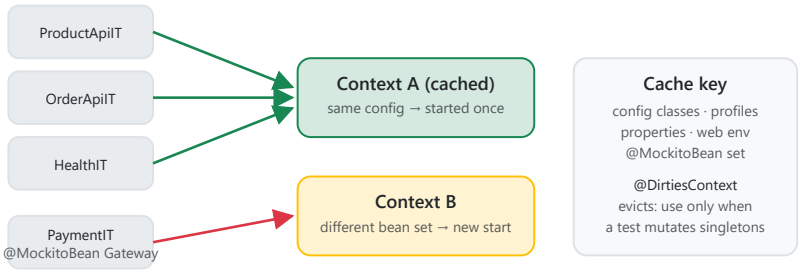
```java
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
class ProductApiIT {
    @Autowired TestRestTemplate rest;   // pre-bound to http://localhost:<port>
    @Autowired RestTestClient client;   // Spring Framework 7 fluent client
    @Autowired JwtService jwt;

    @Test
    void createThenReadProduct() {
        HttpHeaders h = new HttpHeaders();
        h.setBearerAuth(jwt.issue("admin", List.of("ROLE_ADMIN")));
        var body = Map.of("sku", "KB-01", "name", "Keyboard");

        ResponseEntity<ProductDto> created = rest.exchange(
            "/api/products", HttpMethod.POST,
            new HttpEntity<>(body, h), ProductDto.class);

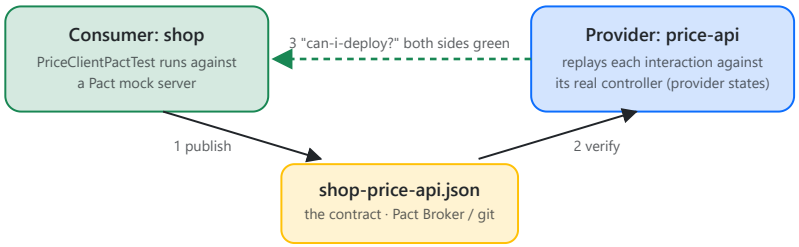
        assertThat(created.getStatusCode()).isEqualTo(HttpStatus.CREATED);
        URI location = created.getHeaders().getLocation();
        ProductDto read = rest.exchange(location, HttpMethod.GET,
            new HttpEntity<>(h), ProductDto.class).getBody();
        assertThat(read.name()).isEqualTo("Keyboard");
    }

    @Test
    void unknownProductIs404ProblemDetail() {
        client.get().uri("/api/products/{id}", 999)
              .header(HttpHeaders.AUTHORIZATION, "Bearer " + userToken())
              .exchange()
              .expectStatus().isNotFound()
              .expectBody().jsonPath("$.detail")
                  .isEqualTo("Product 999 not found");
    }
}
```



Client	Notes
`TestRestTemplate`	classic, tolerant of 4xx/5xx (no exception), `exchange`/`getForEntity`
`RestTestClient`	new in Spring 7: fluent `expectStatus()`, `expectBody().jsonPath()`; also binds to MockMvc
`MockMvc`	with `webEnvironment = MOCK` + `@AutoConfigureMockMvc`: no socket, still the full context

Contract testing: consumer and provider agree without a shared environment



Only the fields the consumer uses are in the contract, so the provider stays free to add fields.

Pact	Spring Cloud Contract
consumer writes the contract in a test (Java DSL)	provider publishes contracts (Groovy/YAML DSL)
broker tracks versions, "can-i-deploy"	generates provider tests and consumer stubs (WireMock)
polyglot (JS, .NET, Go consumers)	Spring-centric, Maven/Gradle plugins

```
// consumer side (sketch): au.com.dius.pact.consumer:junit5
@ExtendWith(PactConsumerTestExt.class)
@PactTestFor(providerName = "price-api")
class PriceClientPactTest {

    @Pact(consumer = "shop")
    V4Pact priceForKnownSku(PactDslWithProvider builder) {
        return builder
            .given("sku KB-01 exists")                // provider state
            .uponReceiving("a price lookup for KB-01")
                .path("/prices/KB-01").method("GET")
            .willRespondWith()
                .status(200)
                .body(new PactDslJsonBody()
                    .stringType("sku", "KB-01")
                    .decimalType("amount", 49.90)
                    .stringType("currency", "EUR"))
            .toPact(V4Pact.class);
    }

    @Test
    @PactTestFor(pactMethod = "priceForKnownSku")
    void consumerCanReadThePrice(MockServer server) {
        var client = new PriceClient(RestClient.create(server.getUrl()));

        assertThat(client.currentPrice("KB-01")).isEqualTo(Money.of("49.90"));
    }
}
```

```
// Spring Cloud Contract equivalent (provider repo, Groovy DSL, sketch)
Contract.make {
    request { method GET(); url '/prices/KB-01' }
    response {
        status OK()
        body(sku: 'KB-01', amount: 49.90, currency: 'EUR')
        headers { contentType(applicationJson()) }
    }
}
```

End-to-end tests (few) and regression tests (one per bug)

```java
// e2e: real browser against a running stack (Playwright for Java)
// com.microsoft.playwright:playwright, run in the "verify" phase
@Tag("e2e")
class ProductPageE2E {
    static final String BASE = System.getProperty("shop.url",
                                                  "http://localhost:8080");

    @Test
    void customerSeesProductAndAddsItToTheCart() {
        try (Playwright pw = Playwright.create()) {
            Browser browser = pw.chromium().launch();
            Page page = browser.newPage();
            page.navigate(BASE + "/products/KB-01");

            assertThat(page.getByRole(AriaRole.HEADING))
                .hasText("Mechanical Keyboard");
            assertThat(page.getByTestId("price")).hasText("49,90 €");

            page.getByRole(AriaRole.BUTTON,
                new Page.GetByRoleOptions().setName("Add to cart")).click();

            assertThat(page.getByTestId("cart-count")).hasText("1");
        }
    }
}
```

- Selenium WebDriver is the classic alternative; Playwright auto-waits for elements and ships browsers, which removes most flakiness.
- Keep e2e tests to the critical journeys (browse, add to cart, pay). Everything else is cheaper one level down.
- Select by role and `data-testid`, never by CSS class or XPath position.

```java
/**
 * Regression test for bug #482.
 * Report: "Ordering the last unit of a product leaves stock at -1
 * and a second customer can still order it."
 * Cause: OrderService checked  stock < quantity  instead of  <=.
 */
@Test
void bug482_orderingTheLastUnitLeavesStockAtZeroAndBlocksTheNext() {
    Product last = aProduct().withStock(1).build();
    when(repo.findBySku(last.sku())).thenReturn(Optional.of(last));
    when(payments.charge(any())).thenReturn(new Receipt("rc"));

    orders.place("alice", last.sku(), 1);

    assertThat(last.stock()).isZero();
    assertThatThrownBy(() -> orders.place("bob", last.sku(), 1))
        .isInstanceOf(InsufficientStockException.class)
        .hasMessageContaining(last.sku());
    verify(payments, times(1)).charge(any());   // bob was never charged
}
```

Regression workflow

1. Reproduce the report as a test at the lowest level that shows it. Watch it fail (red).
2. Fix the code, watch the test pass (green), refactor.
3. Keep the test forever; name it after the ticket so the history is searchable.
4. Ask "which test should have caught this?" and add the missing case class, not just this one.

Code coverage with JaCoCo: measure, then gate

target/site/jacoco/index.html

Element	Missed instr.	Lines	Branches	Missed
edu.itc.shop.service		92 %	85 %	6
edu.itc.shop.web		78 %	60 %	14
edu.itc.shop.security		41 %	30 %	22

Counters: instructions · branches · lines · methods · classes · cyclomatic complexity

OrderService.java (source view)

```
if (product.stock() < quantity) {
throw new InsufficientStockException(..);
}
return stock > 0 && !product.discontinued();
```

green covered · red missed · yellow: some branches missed

What coverage cannot tell you

executed ≠ asserted: a test without assertions covers 100 % of what it calls

100 % lines still misses input combinations

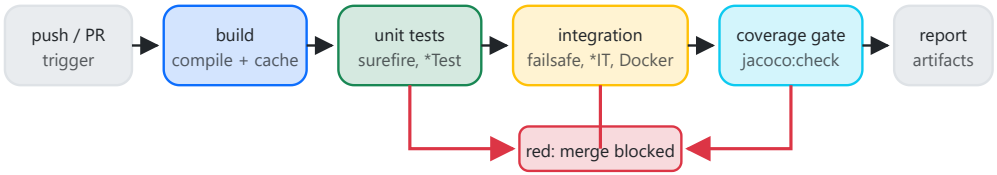
mutation testing (PIT) measures assertions

```xml
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.14</version>
    <executions>
        <execution>
            <!-- sets ${argLine} so Surefire starts the JVM with the agent -->
            <goals><goal>prepare-agent</goal></goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>verify</phase>
            <goals><goal>report</goal></goals>
        </execution>
        <execution>
            <id>coverage-gate</id>
            <phase>verify</phase>
            <goals><goal>check</goal></goals>
            <configuration>
                <rules>
                    <rule>
                        <element>BUNDLE</element>
                        <limits>
                            <limit>
                                <counter>LINE</counter>
                                <value>COVEREDRATIO</value>
                                <minimum>0.80</minimum>
                            </limit>
                        </limits>
                    </rule>
                </rules>
                <excludes>
                    <exclude>**/ShopApplication.class</exclude>
                    <exclude>**/dto/*</exclude>
                </excludes>
            </configuration>
        </execution>
    </executions>
</plugin>
```

Use the gate as a floor that never drops (80 % line, then raise it), not as a target to chase. A `BUNDLE` rule tolerates one weak class; add a `CLASS` rule with a lower minimum to catch untested classes.

Continuous integration: every push runs the whole pyramid

./mvnw -B verify runs all of this in one command, locally and in CI



Branch protection: the "test" job must be green before a pull request can merge into main.

	Surefire	Failsafe
Phase	`test`	`integration-test` + `verify`
Pattern	`*Test`, `Test*`, `*Tests`	`*IT`, `IT*`, `*ITCase`
On failure	stops the build immediately	runs `post-integration-test` cleanup first

```yaml
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest   # Docker preinstalled: Testcontainers works
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '25'
          cache: maven

      - name: Build, test, coverage gate
        run: ./mvnw -B --no-transfer-progress verify

      - name: Upload test and coverage reports
        if: always()              # also when tests failed
        uses: actions/upload-artifact@v4
        with:
          name: reports
          path: |
            target/surefire-reports
            target/failsafe-reports
            target/site/jacoco
```

⚠ CI must be deterministic: no dependency on the developer's local database, wall-clock time, port 8080 being free, or test execution order. If it only fails in CI, the test is the bug.

Maintainable tests: smells to remove, names that explain

Smell	Why it hurts	Fix
`Thread.sleep`	slow when it passes, flaky when the machine is busy	Awaitility `await().untilAsserted(..)`, or make the code synchronous for the test
Order dependence	passes alone, fails in the suite (or the reverse)	fresh fixture per test; rollback; no static mutable state
Over-mocking	test mirrors the implementation; refactoring breaks it	mock only I/O boundaries, assert on outcomes
No assertion	green forever, proves nothing	assert result or interaction; PIT finds these
Logic in tests	loops, ifs, recomputed expectations can be wrong too	literal expected values, parameterized cases
Mystery guest	depends on a file, row or env var nobody sees	builders, `@Sql` next to the test, `@TempDir`
Testing private methods	locks in structure, not behavior	test through the public API or extract a class

Convention	Example
Class per unit, suffix by kind	`ProductServiceTest`, `ProductApiIT`, `ProductPageE2E`
Method = behavior sentence	`createStoresProductWithCurrentPrice()`
or *method_condition_result*	`get_unknownId_throwsNotFound()`
`@DisplayName` for readers	"adds 19 % German VAT to a net price"
Test package mirrors main	`src/test/java/edu/itc/shop/service/`

```java
// smell: sleep and hope
@Test
void eventuallyShipped() throws Exception {
    orders.ship(id);
    Thread.sleep(2000);
    assertEquals(SHIPPED, orders.get(id).status());
}

// fix: poll with a deadline (Awaitility is in spring-boot-starter-test)
@Test
void isShippedWithinTwoSeconds() {
    orders.ship(id);
    await().atMost(Duration.ofSeconds(2))
           .untilAsserted(() ->
               assertEquals(SHIPPED, orders.get(id).status()));
}

// smell: order-dependent tests sharing static state
static Product shared;
@Test @Order(1) void create() { shared = service.create(cmd); }
@Test @Order(2) void read()   { assertNotNull(service.get(shared.id())); }

// fix: each test owns its data
@Test
void readReturnsWhatWasCreated() {
    Product created = service.create(cmd);
    assertThat(service.get(created.id())).isEqualTo(created);
}

// smell: asserting how, not what
verify(repo).findById(1L);
verify(mapper).toDto(any());
verify(repo, times(1)).save(any());
// fix: assert the outcome, verify only the essential side effect
assertThat(dto.name()).isEqualTo("Keyboard");
verify(mailer).sendConfirmation(order.id());
```

BEST PRACTICES

Testing best practices and common mistakes

1 Follow the pyramid

Most tests are unit tests without Spring; a layer of slice and container tests; a handful of end-to-end journeys. Measure the suite time and keep it under a few minutes.

2 One behavior per test

Arrange-Act-Assert, a name that reads as a sentence, and `assertAll` or AssertJ chains when several facts describe one outcome.

3 Mock the boundary, not the world

Repositories, HTTP clients, clocks and message brokers are mocked or faked. Value objects, mappers and the class under test are real.

4 Keep strict stubs on

Mockito's `UnnecessaryStubbingException` is a gift: it points at dead setup. Reach for `lenient()` only in shared `@BeforeEach` stubs.

5 Right slice for the layer

`@WebMvcTest` for controllers and filters, `@DataJpaTest` for repositories, `@RestClientTest` for clients; `@SpringBootTest` only for wiring and a few journeys.

6 Test on the real database

Testcontainers with `@ServiceConnection` runs PostgreSQL in CI; H2 is a fallback for machines without Docker, never the only target.

7 Mistake: chasing 100 % coverage

Coverage tells you what is *not* tested, never that something is well tested. Gate at a sensible floor and use mutation testing to check assertion strength.

8 Mistake: tolerating flaky tests

A test that fails "sometimes" trains the team to ignore red. Fix it the same day, quarantine with `@Disabled("issue #…")` if you must, never retry-until-green.

9 Mistake: many test contexts

Every distinct `@MockitoBean` set or property override starts another Spring context. Share a few test configurations; avoid `@DirtiesContext`.

Chapter quiz 10 questions

1. According to the test pyramid, which kind of test should be the most numerous in the shop project?

- End-to-end browser tests
- Unit tests without Spring
- @SpringBootTest tests on a random port
- Manual exploratory tests

2. In a JUnit Jupiter class with the default lifecycle, which method runs exactly once before all tests and must be static?

- @BeforeEach
- @BeforeAll
- @Nested
- @Test

3. What does assertAll(...) do that a sequence of plain assertEquals calls does not?

- It runs the assertions in parallel threads
- It executes every assertion and reports all failures together
- It stops the test at the first passing assertion
- It retries failed assertions

4. Which annotation feeds a @ParameterizedTest from a static method returning Stream?

- @ValueSource
- @CsvSource
- @MethodSource
- @EnumSource

5. With @ExtendWith(MockitoExtension.class) and default settings, a when(...).thenReturn(...) stub that the test never uses causes...

- nothing, unused stubs are ignored
- a compile error
- an UnnecessaryStubbingException that fails the test
- a warning printed to System.out only

6. What is the difference between a @Spy and a @Mock in Mockito?

- A spy calls the real methods unless stubbed; a mock returns defaults
- A spy cannot be verified
- A mock wraps a real instance; a spy is empty
- There is no difference in Mockito 5

7. In a @WebMvcTest(ProductController.class) slice, how does the test get a ProductService?

- The real @Service bean is component-scanned automatically
- It must be declared with @MockitoBean (or @Import)
- MockMvc creates one on the fly
- It is injected from application.yml

8. Which statement about @DataJpaTest is correct by default?

- It starts the embedded Tomcat server
- Each test runs in a transaction that is rolled back and uses an embedded database
- It commits every test so data can be inspected
- It requires Docker to run

9. What does @ServiceConnection on a Testcontainers PostgreSQLContainer field do?

- Opens a JDBC connection pool inside the test class
- Derives spring.datasource.* properties from the running container automatically
- Replaces PostgreSQL with H2
- Starts the container only when Docker is missing

10. The JaCoCo check goal is configured with counter LINE, value COVEREDRATIO, minimum 0.80. What happens when line coverage is 76 %?

- The build fails in the verify phase
- A warning is logged and the build passes
- Coverage is rounded up to 80 %
- Only the HTML report is skipped

WRAP-UP

Summary

The test pyramid: many fast unit tests, fewer slice and container tests, a handful of end-to-end journeys.

JUnit 6 Jupiter: `@BeforeAll/@BeforeEach/@Test/@AfterEach/@AfterAll`, `@Nested`, `@DisplayName`; Java 17+ and no Vintage engine.

Assert one behavior: `assertAll`, `assertThrows`, and AssertJ's fluent `assertThat(..).extracting(..)`.

`@ParameterizedTest` with `@CsvSource`, `@MethodSource`, `@EnumSource` turns cases into rows; `@RepeatedTest` probes randomness.

Mockito 5: `@Mock`, `when/thenReturn`, `verify`, `ArgumentCaptor`, `@Spy`; strict stubs flag dead setup.

Slices: `@WebMvcTest` + `MockMvcTester` + `@MockitoBean`, `@DataJpaTest` + `TestEntityManager`, filters through the security chain.

Fixtures: builders and Object Mothers keep tests readable; `@Sql` and rollback keep them independent.

Integration: Testcontainers PostgreSQL via `@ServiceConnection`, `MockRestServiceServer` or WireMock for HTTP clients, `@SpringBootTest(RANDOM_PORT)` for the whole API.

Contracts (Pact), regression tests per bug, JaCoCo gate at 80 % in a GitHub Actions job that runs on every push.

Open Lab-06: 5 tasks + 1 challenge

Next chapter: 07 · Introduction to UML and Use Case Diagrams